



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Luna Samantha Arrazola Escobar Carné: 1022725

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Una API REST es una interfaz que permite que dos aplicaciones se comuniquen a través de solicitudes HTTP. Se usa para acceder y trabajar con recursos de un sistema, por ejemplo, para consultar, registrar, modificar o eliminar información.

2. ¿Qué es un endpoint?

Un endpoint es una dirección específica dentro de una API a la que se envía una solicitud para acceder a un recurso o realizar una operación.

3. ¿Qué diferencia existe entre una URL y un endpoint?

Una URL es la dirección completa que se usa para localizar un recurso en Internet. Un endpoint es una ruta específica dentro de una API que permite realizar una operación sobre un recurso. Por ejemplo, la URL puede contener el dominio de la API y el endpoint indica específicamente a qué recurso se quiere acceder.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

Cada método indica la acción que se realizará sobre un recurso: GET consulta información, POST crea un nuevo recurso, PUT actualiza un recurso completo, PATCH modifica solo algunos de sus datos y DELETE elimina un recurso existente.

5. ¿Qué es JSON y por qué es común en las APIs?

JSON es un formato de texto usado para representar e intercambiar datos mediante pares de clave y valor. Es común en las APIs porque es fácil de leer, tiene una estructura sencilla y puede ser usado por diferentes lenguajes de programación.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Un código de estado HTTP es un número que devuelve el servidor para indicar el resultado de una solicitud. 200 OK indica que la solicitud fue exitosa, 404 Not Found que el recurso solicitado no fue encontrado y 500 Internal Server Error que ocurrió un problema interno en el servidor.

7. ¿Qué información se puede encontrar en la respuesta de una API?

En la respuesta de una API se puede encontrar un código de estado HTTP, los datos solicitados generalmente en formato JSON, los headers de respuesta y dependiendo del resultado, mensajes que indican si la operación fue exitosa o si ocurrió algún error.



8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Postman permite probar una API sin tener que crear primero un programa que la consuma. Se pueden configurar métodos HTTP, URLs, parámetros, headers y cuerpos JSON, además de revisar las respuestas y códigos de estado para comprobar que cada operación funcione correctamente.

9. ¿Qué es y para qué sirve Swagger UI?

Swagger UI es una herramienta que presenta de forma visual e interactiva la documentación de una API. Sirve para conocer sus endpoints, métodos HTTP, parámetros y estructuras de datos, además de permitir ejecutar solicitudes directamente desde el navegador.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Postman funciona como una herramienta para crear, enviar y organizar solicitudes hacia una API. Durante el desarrollo permite probar operaciones como GET, POST, PUT y DELETE, revisar las respuestas del servidor y detectar errores antes de integrar la API en una aplicación.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.

El endpoint utilizado para consultar una mascota por su ID es /pet/{petId}.

- Indique el método HTTP utilizado. (Get)

El método HTTP utilizado es GET, ya que se desea consultar información de un recurso existente.

- Indique qué parámetro necesita la solicitud. (PetID)

La solicitud necesita el parámetro petId, que corresponde al ID de la mascota que se quiere consultar. Para esta prueba se usó el valor 1.

- Ejecute la solicitud con Try it out y registre el código HTTP recibido.

Al ejecutar la solicitud mediante Try it out, se recibió el código HTTP 200, lo cual indica que la solicitud se realizó correctamente.

- Explique con sus palabras qué información contiene la respuesta.

La respuesta contiene la información de la mascota con ID 1 en formato JSON. Se muestra su nombre "Pet1", las direcciones asociadas en photoUrls, una lista de etiquetas (tags) y su estado "available", indicando que la mascota está disponible.



Evidencia: captura de Swagger mostrando la operación y su respuesta.

The screenshot shows the Swagger UI for a GET endpoint `/pet/{petId}`. The description states it returns a single pet. A parameter `petId` is defined as a required integer. The response is a 200 status code with a JSON body containing pet details. The response headers include access-control and content-type information.

GET `/pet/{petId}` Find pet by ID.

Returns a single pet.

Parameters

Name	Description
<code>petId</code> *required <code>integer(\$int64)</code> (path)	ID of pet to return

Responses

200

Response body

```
{
  "id": 1,
  "name": "Pet1",
  "photoUrls": [
    "test1",
    "test2"
  ],
  "tags": [],
  "status": "available"
}
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 83
content-type: application/json
date: Sat, 03 Oct 2026 04:25:00 GMT
server: Jetty(9.4.53.v20231009)
```

Responses

Code	Description	Links
200	successful operation	No links

3. Crear un recurso con POST

Utilice POST `/pet` para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET `/pet/{petId}`.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

En las imágenes de a continuación se evidencia que se utilizó el método POST `/pet` para registrar una nueva mascota mediante un JSON propio. Se utilizó el ID 1022725 y se enviaron los atributos solicitados: id, name, status, category y tags. La API devolvió una respuesta exitosa y posteriormente se realizó una solicitud GET utilizando el mismo ID para comprobar que el recurso había sido creado correctamente.



POST /pet Add a new pet to the store.

Add a new pet to the store.

Parameters Cancel Reset

No parameters

Request body **required** application/json

Create a new pet in the store

Edit Value Schema

```
{
  "name": "Molly",
  "category": {
    "id": 1,
    "name": "Perro"
  },
  "tags": [
    {
      "id": 1,
      "name": "Mascota"
    }
  ],
  "status": "available"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1022725,
    "name": "Molly",
    "category": {
      "id": 1,
      "name": "Perro"
    },
    "tags": [
      {
        "id": 1,
        "name": "Mascota"
      }
    ],
    "status": "available"
  }'
```

Request URL
`https://petstore3.swagger.io/api/v3/pet`

Server response

Code Details

200

Response body

```
{
  "id": 1022725,
  "category": {
    "id": 1,
    "name": "Perro"
  },
  "name": "Molly",
  "photoUrls": [],
  "tags": [
    {
      "id": 1,
      "name": "Mascota"
    }
  ],
  "status": "available"
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 135
content-type: application/json
date: Sat, 03 Oct 2026 04:32:04 GMT
server: Jetty(9.4.53.v20231009)
```

Responses

Code	Description	Links
200	Successful operation	No links

Media type
application/json

Controls Accept header.

Example Value Schema

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```



GET /pet/{petId} Find pet by ID.

Returns a single pet.

Parameters

Name	Description
petId • required	ID of pet to return

integer(int64) 1022725 (path)

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'https://petstore3.swagger.io/api/v3/pet/1022725' \
-H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/1022725

Server response

Code	Details
200	Response body <pre>{ "id": 1022725, "category": { "id": 1, "name": "Perro" }, "name": "Molly", "photoUrls": [], "tags": [{ "id": 1, "name": "Mascota" }], "status": "available" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 135 content-type: application/json date: Sat, 03 Oct 2026 04:35:24 GMT server: Jetty(9.4.53.v20231009)</pre>

Responses

Code	Description	Links
200	successful operation	No links

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?

Se utilizó el método HTTP PUT, ya que permite actualizar la información de un recurso existente. En este caso se modificaron el nombre y el estado de la mascota creada anteriormente.

- ¿Qué datos fueron enviados en el body?

En el body se enviaron los datos de la mascota en formato JSON, manteniendo el ID 1022725 y modificando el nombre a Nisha y el estado a pending.

- ¿Qué código HTTP devolvió la API?

La API devolvió el código HTTP 200, lo que indica que la solicitud se realizó correctamente.

- ¿Cómo comprobó que el cambio realmente ocurrió?

Para comprobar el cambio se realizó una solicitud GET /pet/{petId} utilizando nuevamente el ID 1022725. La respuesta mostró el nombre Nisha y el estado pending, confirmando que la información fue actualizada.



PUT /pet Update an existing pet.

Update an existing pet by id.

Parameters

Cancel

Reset

No parameters

Request body **required**

application/json

Update an existent pet in the store

Edit Value Schema

```
{
  "id": 1022725,
  "name": "Nisha",
  "category": {
    "id": 1,
    "name": "Perro"
  },
  "tags": [
    {
      "id": 1,
      "name": "Mascota"
    }
  ],
  "status": "pending"
}
```

Execute

Clear

Responses

Curl

```
-H 'Content-Type: application/json' \
-d '{
  "id": 1022725,
  "name": "Nisha",
  "category": {
    "id": 1,
    "name": "Perro"
  },
  "tags": [
    {
      "id": 1,
      "name": "Mascota"
    }
  ],
  "status": "pending"
}'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code

Details

200

Response body

```
{
  "id": 1022725,
  "category": {
    "id": 1,
    "name": "Perro"
  },
  "name": "Nisha",
  "photoUrls": [],
  "tags": [
    {
      "id": 1,
      "name": "Mascota"
    }
  ],
  "status": "pending"
}
```

Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 133
content-type: application/json
date: Sat, 03 Oct 2026 04:48:44 GMT
server: Jetty(9.4.53.v20231009)
```

Responses

Code

Description

Links

200

Successful operation

Media type

application/json

Controls Accept header.

Example Value Schema

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "string"
}
```

No links

400

Invalid ID supplied

No links

404

Pet not found

No links



GET /pet/{petId} Find pet by ID.

Returns a single pet.

Parameters

Name	Description
petId • required	ID of pet to return
integer(\$int64)	
(path)	1022725

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/1022725' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/1022725
```

Server response

Code	Details
200	Response body <pre>{ "id": 1022725, "category": { "id": 1, "name": "Perro" }, "name": "Nisha", "photoUrls": [], "tags": [{ "id": 1, "name": "Mascota" }], "status": "pending" }</pre> Download Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 133 content-type: application/json date: Sat, 03 Oct 2026 04:43:39 GMT server: Jetty(9.4.53.v20231009)</pre>

Responses

Code	Description	Links
200	successful operation	No links

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.

Se realizó una solicitud GET /pet/findByStatus utilizando el valor available para consultar las mascotas que se encuentran disponibles. La solicitud devolvió un código HTTP 200, indicando que se ejecutó correctamente.

- Identifique el parámetro utilizado.

El parámetro utilizado es status, el cual permite filtrar las mascotas de acuerdo con su estado. Para esta consulta se utilizó el valor available.

- Explique dónde se envía ese parámetro.

El parámetro status se envía en la URL como un parámetro de consulta. En la solicitud realizada aparece después del signo ? de la siguiente manera: ?status=available.

- Observe la estructura de la respuesta.

La respuesta se presenta en formato JSON como una lista de objetos. Cada objeto corresponde a una mascota e incluye datos como su ID, categoría, nombre, fotografías, etiquetas y estado.



- Indique qué sucede si cambia el valor del parámetro por pending.

Al cambiar el valor del parámetro de available a pending, la API hace la misma consulta pero filtra los resultados y devuelve solo las mascotas cuyo estado es pending. La solicitud también devolvió un código HTTP 200, indicando que se ejecutó correctamente.

GET /pet/findByStatus Finds Pets by status.

Multiple status values can be provided with comma separated strings.

Parameters

Name	Description
status *required string (query)	Status values that need to be considered for filter available

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \
-H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available
```

Server response

Code Details

200

Response body

```
[
  {
    "id": 4,
    "category": {
      "id": 1,
      "name": "Dogs"
    },
    "name": "Dog 1",
    "photoUrls": [
      "url1",
      "url2"
    ],
    "tags": [
      {
        "id": 1,
        "name": "tag1"
      }
    ]
  }
]
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 390
content-type: application/json
date: Sat, 03 Oct 2026 04:51:15 GMT
server: Jetty(9.4.53.v20231009)
```

GET /pet/findByStatus Finds Pets by status.

Multiple status values can be provided with comma separated strings.

Parameters

Name	Description
status *required string (query)	Status values that need to be considered for filter pending

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending' \
-H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending
```

Server response

Code Details

200

Response body

```
[
  {
    "id": 1,
    "category": {
      "id": 2,
      "name": "Cats"
    },
    "name": "Cat 3",
    "photoUrls": [
      "url1",
      "url2"
    ],
    "tags": [
      {
        "id": 1,
        "name": "tag3"
      }
    ]
  }
]
```

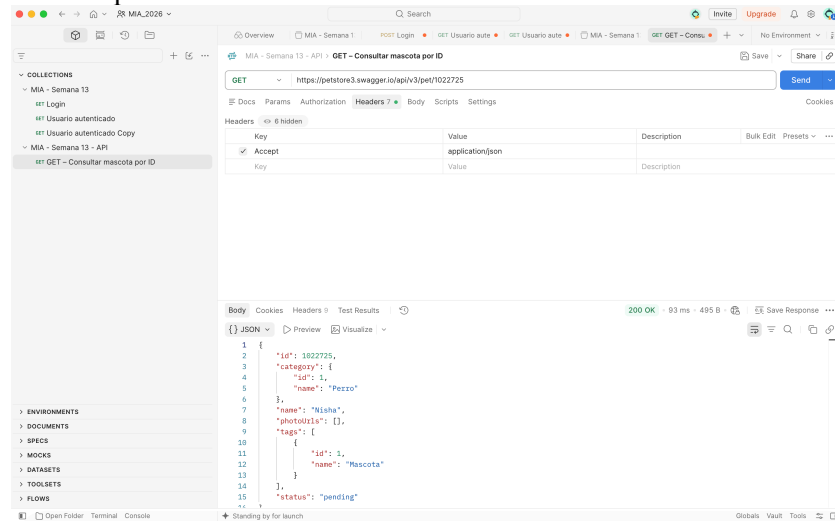
Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 746
content-type: application/json
date: Sat, 03 Oct 2026 04:55:35 GMT
server: Jetty(9.4.53.v20231009)
```

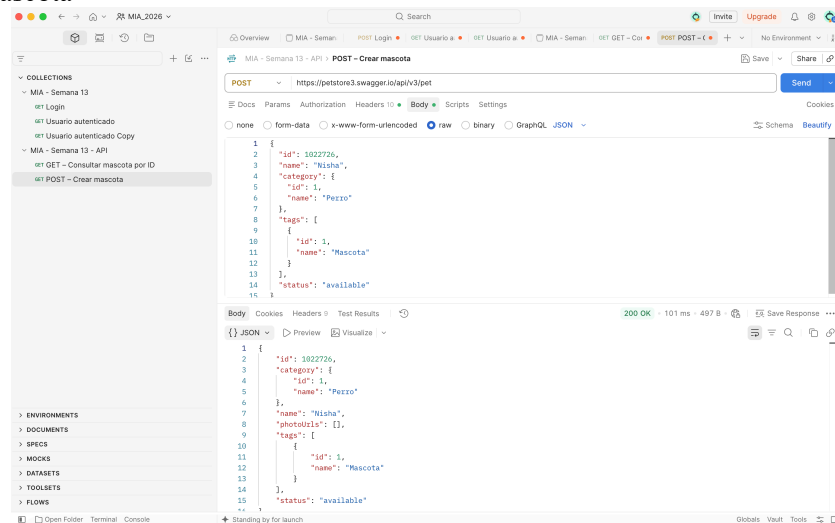


6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID



- POST – Crear mascota





- PUT – Modificar mascota

The screenshot shows the Swagger UI interface for a REST API. The selected endpoint is `PUT https://petstore3.swagger.io/api/v3/pet`. The request body is a JSON object representing a pet with the following structure:

```
1 {
2   "id": 1022726,
3   "name": "Nisha Modificada",
4   "category": {
5     "id": 1,
6     "name": "Perro"
7   },
8   "tags": [
9     {
10      "id": 1,
11      "name": "Mascota"
12    }
13  ],
14  "status": "pending"
15 }
```

The response status is `200 OK` with a response time of 513 ms and a body size of 506 B. The response body is a JSON object with the same structure as the request body.

- GET – Buscar por status

The screenshot shows the Swagger UI interface for a REST API. The selected endpoint is `GET https://petstore3.swagger.io/api/v3/pet/findByStatus/status=pending`. The request headers are set to `Accept: application/json`. The response status is `200 OK` with a response time of 474 ms and a body size of 1.22 KB. The response body is a JSON object representing a pet with the following structure:

```
1 {
2   "id": 3,
3   "category": {
4     "id": 2,
5     "name": "Cats"
6   },
7   "name": "Cat 3",
8   "photoUrls": [
9     "url1",
10    "url2"
11  ],
12  "tags": [
13    {
14      "id": 1,
15      "name": "Mascota"
16    }
17  ]
18 }
```

- DELETE – Eliminar mascota

The screenshot shows the Swagger UI interface for a REST API. The selected endpoint is `DELETE https://petstore3.swagger.io/api/v3/pet/1022726`. The request headers are set to `Accept: application/json`. The response status is `200 OK` with a response time of 515 ms and a body size of 506 B. The response body is a JSON object representing a pet with the following structure:

```
1 {
2   "id": 1022726,
3   "category": {
4     "id": 1,
5     "name": "Perro"
6   },
7   "name": "Nisha Modificada",
8   "photoUrls": [
9     "url1",
10    "url2"
11  ],
12  "tags": [
13    {
14      "id": 1,
15      "name": "Mascota"
16    }
17  ],
18  "status": "pending"
19 }
```



- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

Prueba 1: Consultar un ID inexistente

- a) Solicitud realizada

Se realizó una solicitud GET al endpoint `/pet/{petId}` utilizando el ID 999999999, con el propósito de consultar una mascota que no se encuentra registrada.

- b) Código HTTP recibido

Se recibió el código HTTP 404 Not Found.

- c) Respuesta de la API

La API respondió con el mensaje "Pet not found".

- d) Qué esperaba que ocurriera

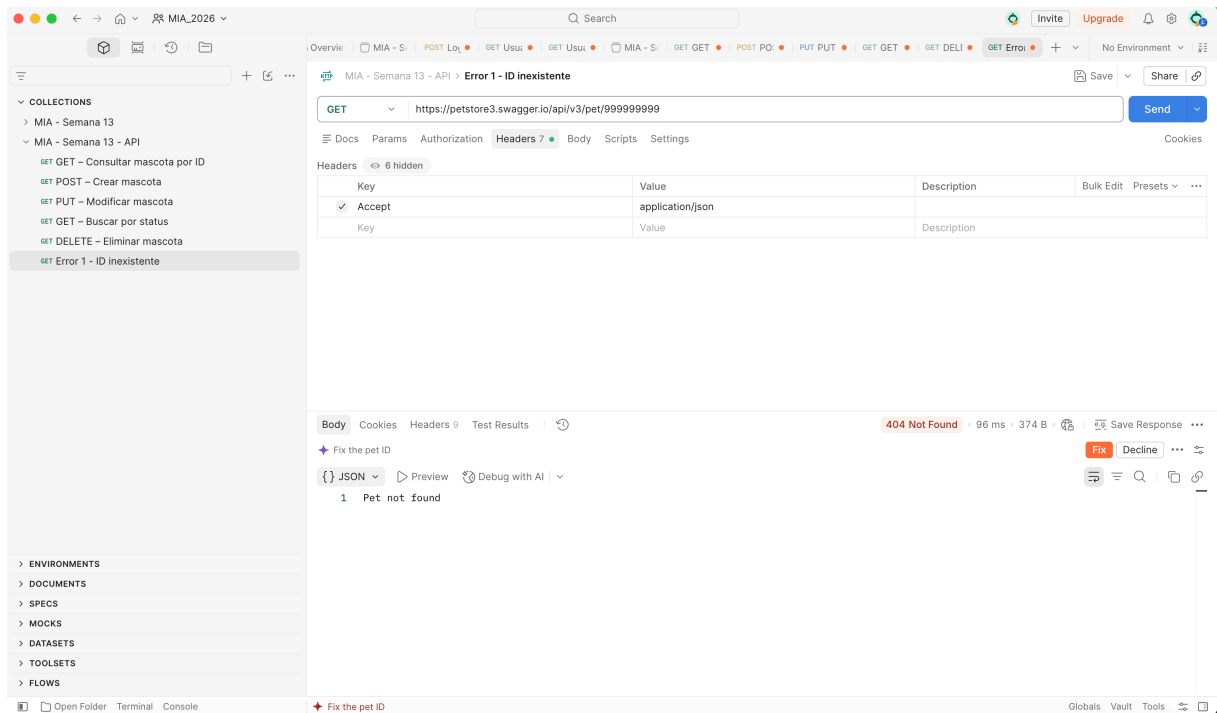
Esperaba que la API indicara que la mascota no fue encontrada, debido a que se utilizó un ID inexistente.

- e) Qué ocurrió realmente

La API devolvió el código 404 Not Found y el mensaje "Pet not found", indicando que no encontró una mascota asociada al ID enviado.

- f) Explicación de la causa

La respuesta ocurrió porque el ID 999999999 no corresponde a una mascota registrada en la API. Por esta razón, el servidor no pudo encontrar el recurso solicitado.



Prueba 2: Enviar un JSON incorrecto o incompleto

a) Solicitud realizada

Se realizó una solicitud POST al endpoint /pet, enviando un body en formato JSON con un error de sintaxis. Se omitió intencionalmente la coma entre los atributos name y status.

b) Código HTTP recibido

Se recibió el código HTTP 400 Bad Request.

c) Respuesta de la API

La API devolvió el código 400 junto con el mensaje: “Input error: unable to convert input to io.swagger.petstore.model.Pet”

d) Qué esperaba que ocurriera

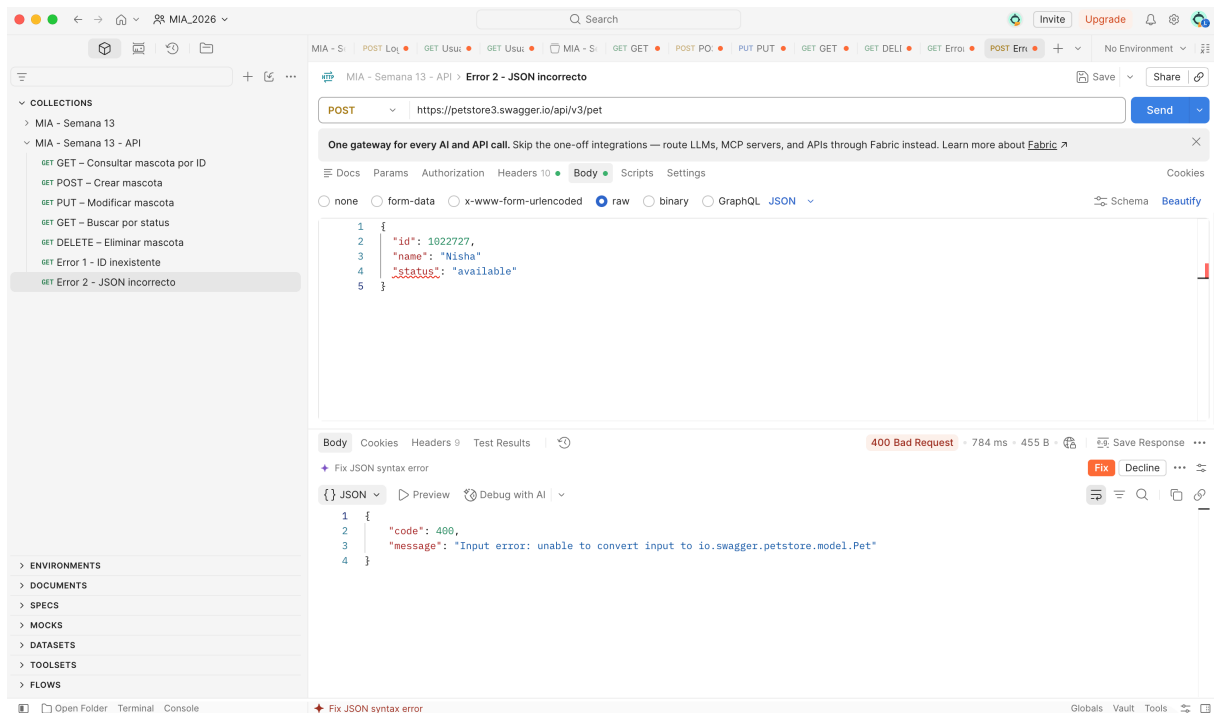
Esperaba que la API rechazara la solicitud, ya que el JSON enviado no tenía una estructura válida.

e) Qué ocurrió realmente

La API rechazó la solicitud y devolvió un código 400 Bad Request, indicando que no pudo convertir los datos enviados al formato esperado para una mascota.

f) Explicación de la causa

El error ocurrió porque faltaba una coma entre los atributos name y status, por lo que el body enviado no cumplía con la sintaxis correcta de JSON y la API no pudo procesarlo.



Prueba 3: Utilizar un parámetro con un valor no esperado

a) Solicitud realizada

Se realizó una solicitud GET al endpoint `/pet/findByStatus` utilizando el parámetro `status` con el valor eliminado.

b) Código HTTP recibido

Se recibió el código HTTP 400 Bad Request.

c) Respuesta de la API

La API devolvió el código 400 e indicó que el valor eliminado enviado en el parámetro `status` no se encuentra entre los valores permitidos: `available`, `pending` y `sold`.

d) Qué esperaba que ocurriera

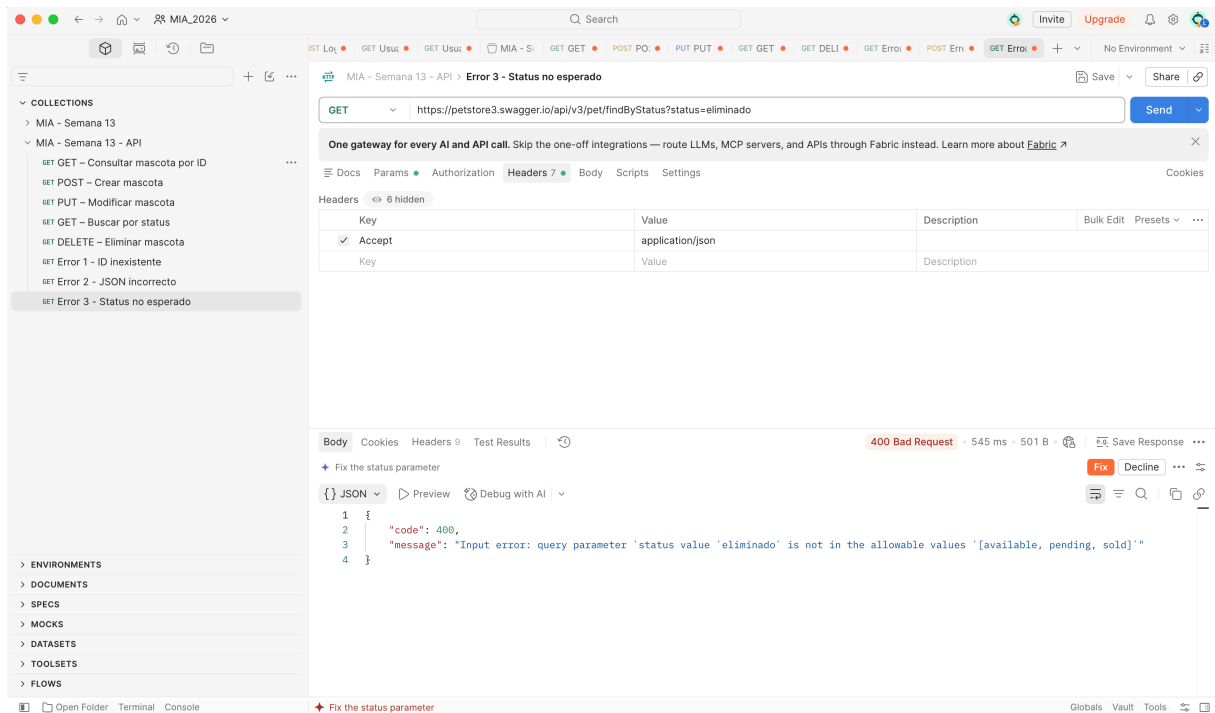
Esperaba que la API rechazara la solicitud porque `eliminado` no corresponde a uno de los valores válidos para el estado de una mascota.

e) Qué ocurrió realmente

La API rechazó la solicitud y devolvió un código 400 Bad Request, indicando que el valor enviado no era válido y mostrando los valores que sí son permitidos.

f) Explicación de la causa

El error ocurrió porque el parámetro `status` únicamente acepta los valores `available`, `pending` y `sold`. Al enviar `eliminado`, la solicitud no cumplió con los valores definidos por la API.



8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

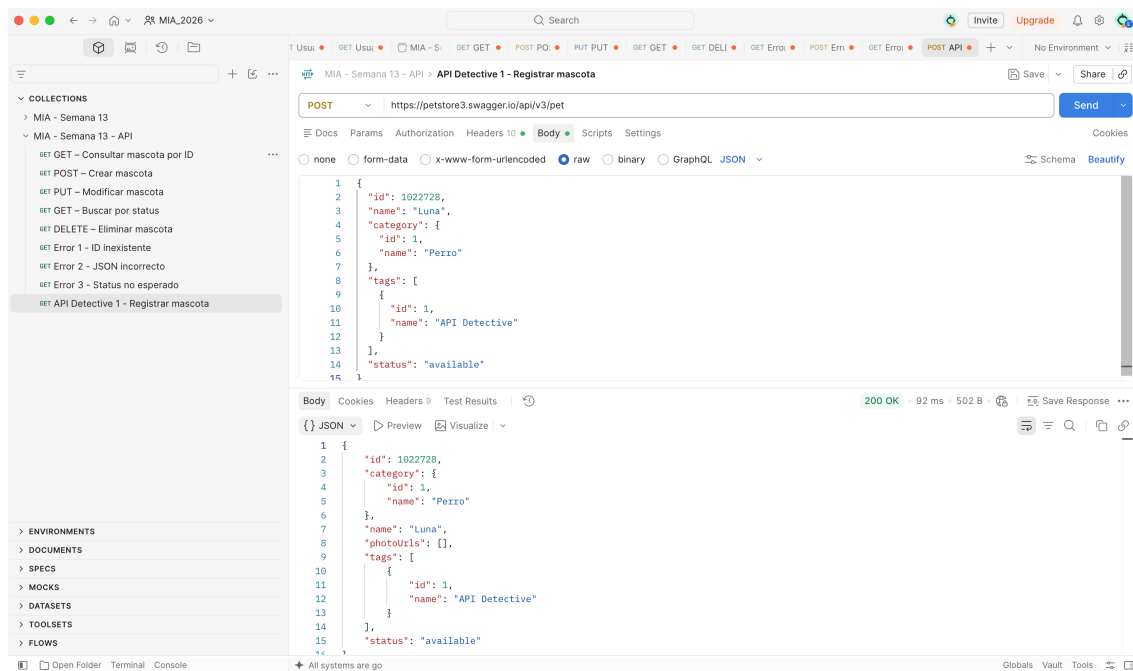
Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

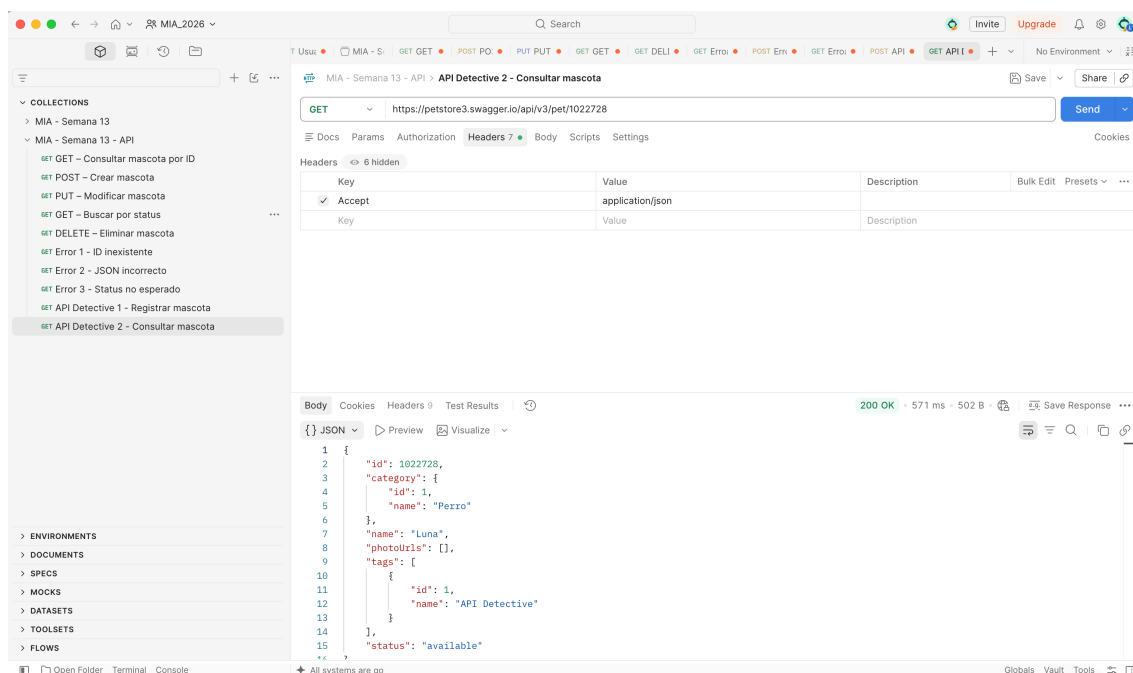
Paso 1: Registrar la mascota

- Endpoint: /pet
- Método HTTP: POST
- Descripción: Se usó el endpoint /pet con el método HTTP POST para registrar una nueva mascota. Se envió un JSON con el ID 1022728, nombre Luna, categoría Perro, una etiqueta llamada API Detective y el estado inicial available.
- Respuesta que tiene que analizar el programa: El programa debe analizar el código de estado HTTP y el contenido de la respuesta para comprobar si la mascota fue registrada correctamente. En esta prueba, la API devolvió el código 200 OK y respondió con los datos de la mascota creada, confirmando que el registro se realizó correctamente.



Paso 2: Consultar la mascota

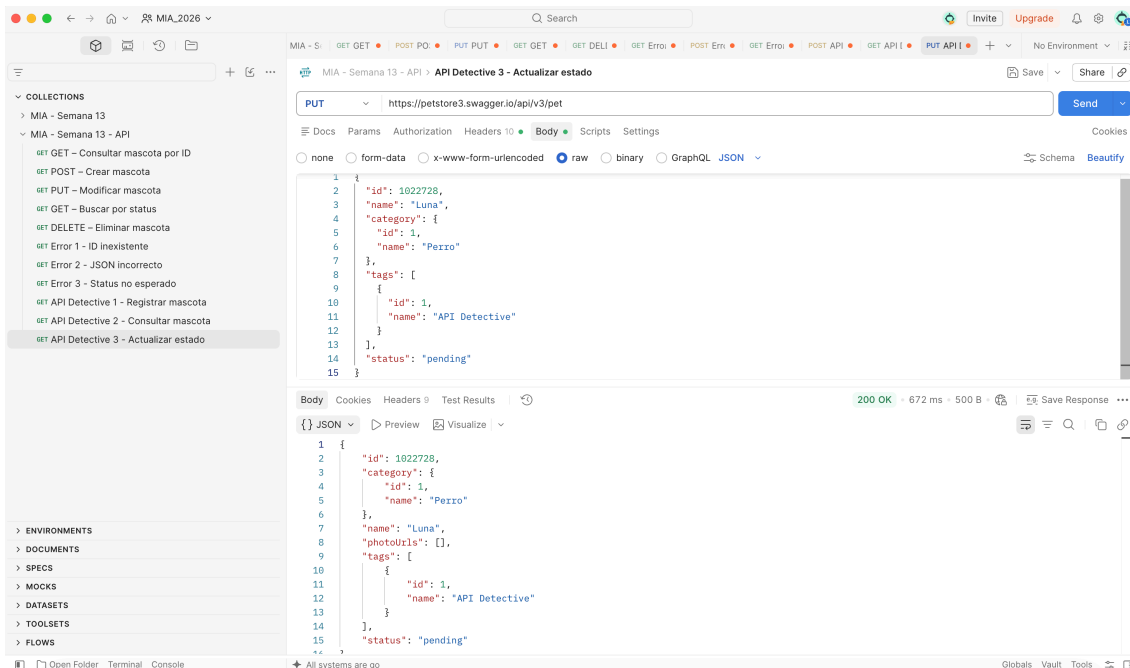
- Endpoint: /pet/{petId}
- Método HTTP: GET
- Descripción: Se usó el endpoint /pet/{petId} mediante el método HTTP GET para consultar la mascota registrada anteriormente. Se utilizó el ID 1022728 para identificar el recurso.
- Respuesta que tiene que analizar el programa: El programa debe analizar el código de estado HTTP y los datos devueltos por la API para determinar si la mascota existe. En esta prueba se recibió el código 200 OK y la respuesta mostró la información de la mascota con ID 1022728, nombre Luna y estado available, confirmando que el recurso fue encontrado correctamente.





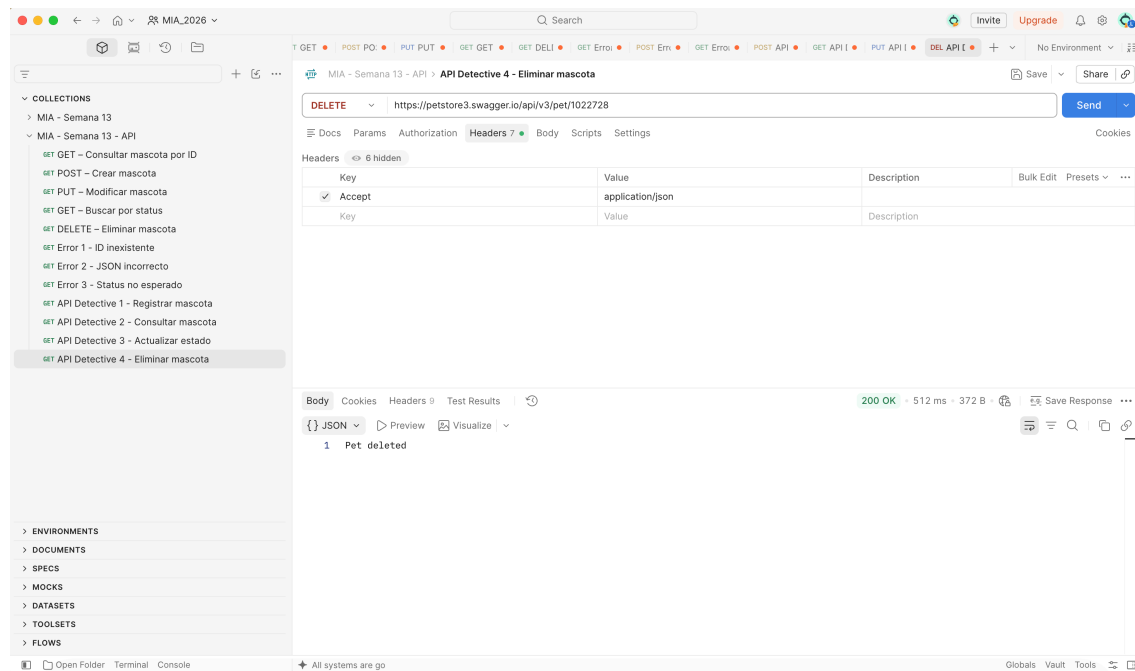
Paso 3: Actualizar el estado

- Endpoint: /pet
- Método HTTP: PUT
- Descripción: Se usó el endpoint /pet mediante el método HTTP PUT para actualizar el estado de la mascota registrada. Se mantuvieron los datos de la mascota con ID 1022728 y se modificó únicamente su estado de available a pending.
- Respuesta que tiene que analizar el programa: El programa debe analizar el código de estado HTTP y los datos devueltos por la API para comprobar que la actualización se realizó correctamente. En esta prueba se recibió el código 200 OK y la respuesta mostró el estado pending, confirmando que la información fue actualizada.



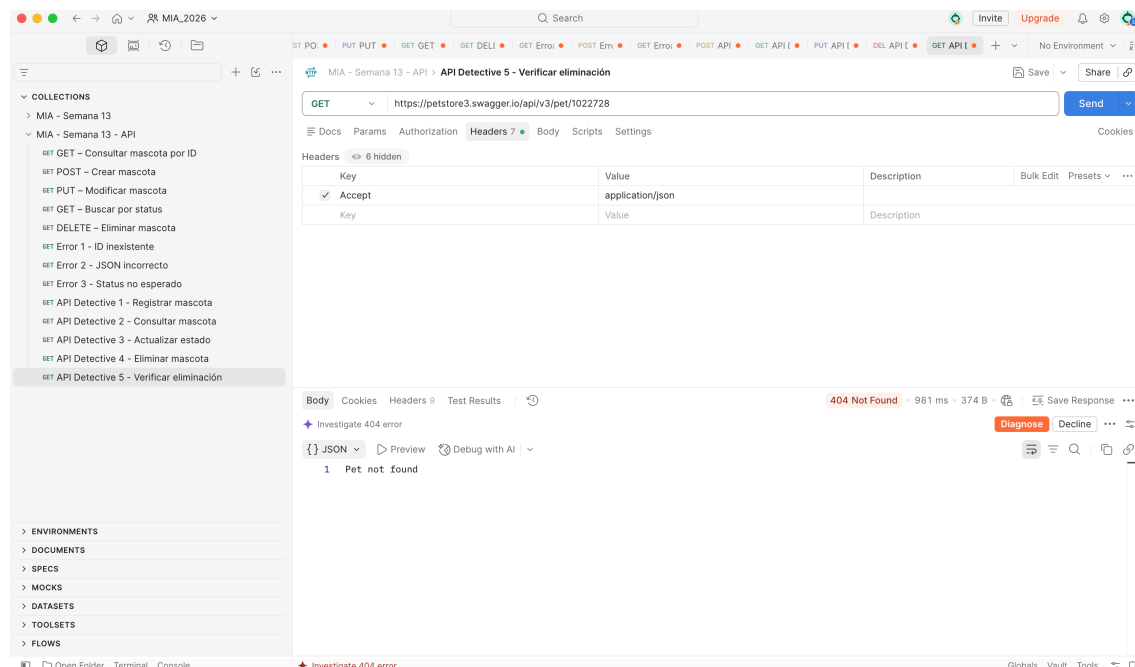
Paso 4: Eliminar la mascota

- Endpoint: /pet/{petId}
- Método HTTP: DELETE
- Descripción: Se usó el endpoint /pet/{petId} mediante el método HTTP DELETE para eliminar la mascota con ID 1022728.
- Respuesta que tiene que analizar el programa: El programa debe analizar el código de estado HTTP y la respuesta de la API para comprobar si la mascota fue eliminada correctamente. En esta prueba se recibió el código 200 OK y el mensaje "Pet deleted", confirmando que el recurso fue eliminado.



Paso 5: Identificar que ya no existe

- Endpoint: `/pet/{petId}`
- Método HTTP: GET
- Descripción: Después de eliminar la mascota, se realizó nuevamente una solicitud GET al endpoint `/pet/{petId}` utilizando el ID 1022728, con el objetivo de comprobar que el recurso ya no se encontraba registrado.
- Respuesta que tiene que analizar el programa: El programa debe analizar el código de estado HTTP para determinar si la mascota existe. En esta prueba, la API devolvió el código 404 Not Found junto con el mensaje “Pet not found”, lo que permite identificar que la mascota con ID 1022728 ya no existe.





Conclusión del API Detective

Con este flujo se comprobó el funcionamiento de las principales operaciones de una API REST. Primero se registró una mascota mediante POST, luego se consultó con GET, se actualizó su estado mediante PUT y se eliminó utilizando DELETE. Por último, se realizó una nueva consulta y se obtuvo un código 404 Not Found lo que confirmó que el recurso ya no existía.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio. Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.